

Prompting and task execution — Principles and critical tips

Pair: [prompting-task-execution-references.md](#)

Course: Associate · Module 2 · Prompting and Task Execution

Related: Module 1 entry points and capability layer under AI/

Goals this note serves: AI as operator · bug bounty / appsec · audit work · architect judgment

Scope: authorized study and authorized testing only

One-sentence definition

An **effective prompt** states role, context the model cannot see, one clear task, constraints, and output format. **Task execution** splits large work into checkable steps. It then fixes weak output by the component that failed.

Why it exists

Default prompts leave gaps. The model fills gaps with generic assumptions. The result sounds fine and still fails the job. Named components turn prompting into a pre-send checklist.

Decomposition stops one huge request from shallow work on every stage. Targeted iteration fixes the broken component. It does not throw away the draft that already works.

Mechanism (first principles)

1. The model only uses text and allowed connectors you supply. Hidden facts are **context gaps**.
2. Each component steers a different failure class.
3. Role sets voice and assumptions. Context fights generic output. Task sets the action. Constraints set length and tone. Format sets shape.
4. A complex job is several tasks in one sentence. You split stages so each stage produces a checkable intermediate result.
5. Weak output is a diagnostic signal. Map the symptom to one component. Change that component. Resend.
6. Task type moves the dial. Analysis and research want tight control. Drafting wants medium latitude. Brainstorming wants range first, then a later cut.

High-level map

```
intent
  → component stack (role / context / task / constraints / format)
  → if multi-stage: decompose (sequence or parallel)
  → model output
  → diagnose missing component
  → targeted fix (stop when change is only marginal)
```

Part	Job
Component stack	Pre-send checklist for non-trivial prompts
Description habit	Make each needed component explicit

Part	Job
Decomposition	Split multi-stage work into ordered, checkable steps
Parallel / separate chat	Independent work or long-context decay
Iteration	Fix the one weak component. Keep what works.
Task-type strategy	Tighten control or open latitude by job class

Fundamental unit: one **context gap** or one **ambiguous instruction**.

Mechanism: close the gap in the prompt or in a prior step. Do not rely on endless full rewrites.

The five components (component stack)

Component	What it controls	Typical miss
Role	Who the model is for this task: vocabulary, depth, assumptions	Generic helper voice on specialist work
Context	Background the model cannot know: audience, situation, prior decisions, source material	Most common miss for professionals
Task	One primary action verb: summarize, compare, draft, identify	Ambiguous or multi-action mush
Constraints	Length, tone, include list, exclude list	Usable draft becomes long edit debt
Output format	Table, bullets, memo shape, email shape	Extra iteration only to reshape

Not every prompt needs all five. A quick question needs a task and often one constraint. A client or report deliverable needs all five. The skill is to see which components the task requires.

Description competency

Description means you state each component. You do not assume the model will infer it.

Without a connector, the model does not see your inbox, org chart, or last meeting. With a connector, it sees only allowed sources. Anything only in your head is a context gap. Context gaps are the most common reason a non-trivial prompt underperforms for new users.

Worked build (same model, different specification)

Weak (everything implicit):

Write a summary of our quarterly operations.

Result: plausible paragraphs for almost any company. No audience. No figures. No format. No priority. Not always false. Still unusable.

Strong (components explicit):

You are an operations analyst (role).
I prepare a one-page update for our regional director.
The director cares about throughput and cost, not process detail (context + audience).
Summarize the attached Q3 operations data (task).
Cover only the three metrics that moved more than 10 percent against target (constraint).
Format as one short headline and three one-sentence bullets (output format).

Same model and data. The second draft needs light polish, not a rebuild. The gain is specification, not luck.

Habit before any prompt that matters: run the five components in your head for about thirty seconds.

Task decomposition

Definition: split a multi-part problem into discrete, ordered steps. Run them in sequence. Use parallel only when independent. Do not ask for everything at once.

Why one packed prompt fails: the model invents criteria, scores, weighs trade-offs, and recommends in one pass. Every stage stays shallow. You cannot audit the reasoning.

Vendor evaluation pattern (four tasks in one sentence)

Step	Job	Checkable output
1	Derive criteria from requirements. Weigh them.	Criteria list with weights
2	Score vendors against those criteria	Score table
3	Raise trade-offs	Explicit trade-off notes
4	Recommend	Recommendation tied to steps 1-3

If step 1 is wrong, you catch it before scoring. Decomposition makes the work auditable when someone asks how you reached the pick.

One conversation or several

Case	Do this
Steps build on prior results	Keep them in one conversation so later steps see earlier output
Steps are independent	Use separate conversations (or parallel runs), then merge
Thread is long and early context degrades	Start a new conversation with a tight handoff summary

This reuses Module 1 context judgment: sticky background vs disposable thread.

Parallel case

Parallel means steps do **not** depend on each other.

Pattern	Example
Parallel research legs	Vendor A facts in chat 1. Vendor B in chat 2. Merge scores later.

Pattern	Example
Parallel drafts	Two tone variants of the same memo in two chats. Pick one.
Forced sequence	Criteria before scores. Scores before final recommend.

Rule: if step B needs step A output, keep sequence. If not, parallel saves time and keeps each thread clean.

Iterating to improve output

Do not rewrite the whole prompt when output disappoints. Read the output. Name the weak component. Fix that one thing.

Symptom	Likely cause	Fix
Generic or off-base	Thin context	Add background the model cannot infer
Wrong question answered	Ambiguous task verb	Sharpen the one instruction
Wrong length, tone, or shape	Missing constraint or format	Add the missing boundary
Mostly right. One section weak.	Local gap	Iterate only that section

Targeted revision

A full rewrite hides which change helped. It also discards working parts. Change one component. Resend. Compare.

Live cycle (compressed)

1. Round 1: "Write a follow-up email about the delayed deliverable." Result: generic, no date, no cause.
2. Round 2: add cause, new date, tone (accountable, not over-apologetic), under 120 words. Result: strong body. Subject still missing.
3. Round 3: add subject line that signals resolution. Then stop.

When to stop

Iteration **converges** when extra rounds only change margins. Then a short manual edit beats more prompting. Goal is a usable result, not a perfect prompt string.

Strategy by task type

The component stack stays. Emphasis moves.

Task type	Tighten	Loosen	Latitude
Analysis	Criteria, standards, scope, ambiguity rules	Phrasing	Low creative. High specification.
Research	Question, boundaries, source rules, citations	How it synthesizes	Scope and source discipline

Task type	Tighten	Loosen	Latitude
Drafting	Audience, tone, format, length	Exact wording	Medium
Brainstorming	Goal and guardrails only	Quantity and direction	High. Constrain after range exists.

Mini patterns

Analysis: compare two contracts on payment terms, termination, and liability. Use a three-row table. State which side favors you and why. No wander.

Research: use current sources for three named competitors. Cite each source. Flag unverified claims. Use web search or Research for currency. Citations from training memory alone need independent verify.

Drafting: 150-word post. Audience fixed. Tone fixed. Phrasing open.

Brainstorming: 20 angles. Range wide. No self-edit yet. Cut later.

Underlying move: decide where you need control and where you need range. Set constraints to match.

Checkpoint answers (diagnose the prompt)

Prompt	Dominant weakness	Single high-leverage change
"Make this better." + draft email	Task (and often constraints or format)	State action and success bar: "Shorten to 100 words. Keep the ask. Cut apology."
"Give me everything you know about supply chain risk."	Constraints + context	Bound scope and use: "Top five risks for a SaaS buyer in EU. One line each. Cite if using web."
"Write a professional document about the project."	Context + task + format	Name audience, purpose, and shape: "2-page status for sponsor: risks, decisions needed, next 30 days."
"Brainstorm names, but one word, under eight letters, avoid twelve terms, match exact brand voice..."	Constraints too tight for brainstorm	Loosen first pass: "20 name candidates. No self-filter. Apply brand rules in pass two."

Critical points everyone misses

1. **Context is the professional miss.** Role and format feel advanced. Missing audience and prior decisions make output generic.
2. **One primary task verb.** Multi-action mush ("evaluate and decide and write the deck") forces shallow work on every verb.
3. **Weak output maps to a component.** If you cannot name the gap, you rewrite at random.
4. **Decomposition is audit, not bureaucracy.** Checkable intermediates protect you when a lead asks "why this vendor?"
5. **Brainstorm and analysis are opposite dials.** Over-constrain brainstorm and under-constrain analysis both fail.

6. **Stop at diminishing returns.** Perfect prompt theater wastes more time than a two-minute manual edit.
7. **Connectors do not equal full world access.** Allowed sources only. The rest is still a context gap.

Gotchas

If you see...	Then...
Fluent generic prose	Add context and audience. Do not only add "be specific".
Confident citations with no search or Research	Verify independently. Treat as ungrounded until checked.
One mega-prompt for a multi-stage decision	Decompose. Fix criteria before scores.
Full rewrite every round	Change one component. Keep working text.
Long thread, worse answers	New chat plus handoff summary of decisions and facts.
Brainstorm with ten hard constraints	Remove constraints. Apply them in a second pass.

Critical tips for operator goals

Goal	Critical tip
Bug bounty / appsec	Role = senior appsec. Context = asset type, auth model, in-scope. Task = one class (for example BOLA paths). Constraints = no out-of-scope exploit advice. Format = repro steps plus impact. Decompose: surface map → hypothesis → test plan → report draft.
Cloud / Azure-style audit	Context must include subscription boundary, identity model, and the baseline (CIS or internal). Analysis style: explicit criteria table before findings narrative.
AI as operator	Pre-send the five components for any non-trivial ask. Prefer decompose plus evidence files over one "do the whole engagement" prompt.
Architect judgment	Force a trade-off step before recommend. Require criteria weights. Reject single-shot "which design is best?" without stated quality bars.

IF / THEN

IF	THEN
Deliverable goes to a real stakeholder	Use all five components
Job has two or more distinct stages	Decompose. Check intermediate output.
Steps do not share results	Parallel or separate chats, then merge
Output is wrong in one way	Fix that component only
Extra rounds change little	Stop prompting. Edit by hand.
Task is brainstorm	Loosen constraints first
Task is analysis	Tighten criteria and output shape

Do this (minimal practice)

1. Take one real prompt you sent this week.

2. Label role, context, task, constraints, format. Mark missing items as gap.
3. Rewrite once with only the missing components filled.
4. If the job is multi-stage, write a 3-5 step decompose plan with a checkable artifact per step.
5. On the next weak output, name the failing component before you type a fix.

Out of scope here

- Official documentation tables → see **references** pair file
 - Product UI tour of Claude plans → Module 1 notes or references
 - Module 7 deep workflow troubleshooting → later module notes
-

Module takeaways (operator card)

1. Name the five components. Use them as pre-send and post-fail checklist.
2. Description beats inference. Close context gaps yourself.
3. Decompose multi-stage work. Keep intermediate results checkable.
4. Iterate one component at a time. Stop when returns diminish.
5. Match control vs latitude to task type (analysis, research, drafting, brainstorm).